F0 / F1 | crystallize_executable_capability | Google DeepMind

让 Flash 先给自己写一道合法性闸门：合成 `is_legal_action()`，再拿它把关

AutoHarness：用环境返回的“非法动作”反馈，树搜索迭代改写动作合法性代码；145 个 TextArena 游戏在新随机种子上的合法率都练到 100%。被比较的是同一个小模型加不加这道闸门，以及更大的 Gemini-2.5-Pro。

Lou, Lázaro-Gredilla, Dedieu, Wendelken, Lehrach, Murphy · arXiv 2603.03329v1 · 2026-02-10

01 / 问题 模型懂规则，却常常走出环境不允许的动作

摘要、§1、§4、附录 A.4–A.5

在棋类和其他文字游戏里，**非法动作不是“走得差”，而是直接判负或卡死**。常见补救是微调（慢、贵、可能伤及其他能力）或为每个游戏手写合法性检查（每来一个新游戏就要重写）。作者的做法：让 LLM 自己写这段检查代码，用环境反馈反复修，直到在没见过的随机种子上不再出非法动作。

78% 作者引用：Kaggle GameArena 国际象棋赛中 Gemini-2.5-Flash 的败局有 78% 归因于非法走子。**此数来自外部赛事报告，本项目未核对。**

实验先把提示里的“合法走法”列表删掉（附录 A.4 原文节选）

```
[GAME] You are playing White in a game of Chess.
       Make your moves in UCI format ... [e2e4].
[GAME] Current board:
 8 | r n b q k b n r |      2 | P P P P P P P P |
 7 | p p p p p p p p |      1 | R N B Q K B N R |
   ...
Valid moves: [g1h3], [g1f3], [b1c3], ... [a2a4]
```

删去，否则代码可直接照抄

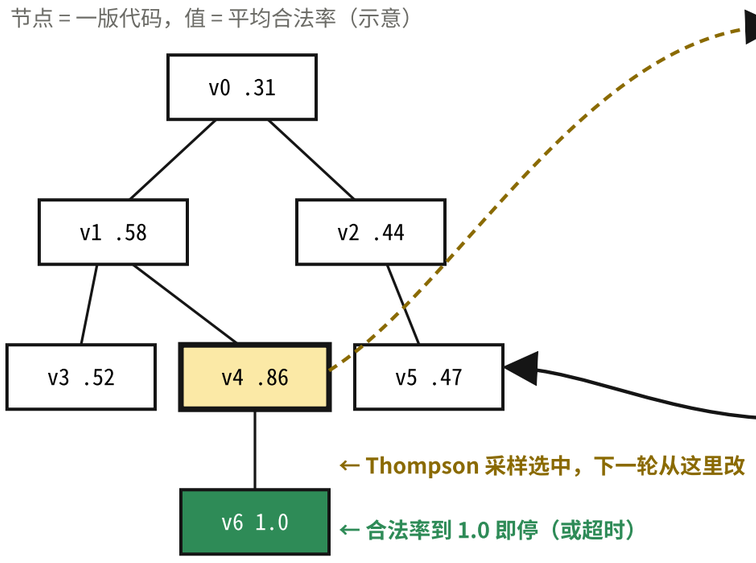
02 / 机制 左边合成闸门，右边部署闸门

§3、§4.1、附录 B.2、C.1 · 树中数值为示意

合成阶段（训练）· 每个游戏单独做一次 · Gemini-2.5-Flash

代码假设树

节点 = 一版代码，值 = 平均合法率（示意）



兼顾探索（换一种代码结构）
与利用（修补已部分可用的版本）

① **10 个并行环境跑 rollout**
每次最多 1000 步，局终自动重置

② **出现非法动作或代码报错 → 立即终止**
环境给出“这一步不合法”的反馈

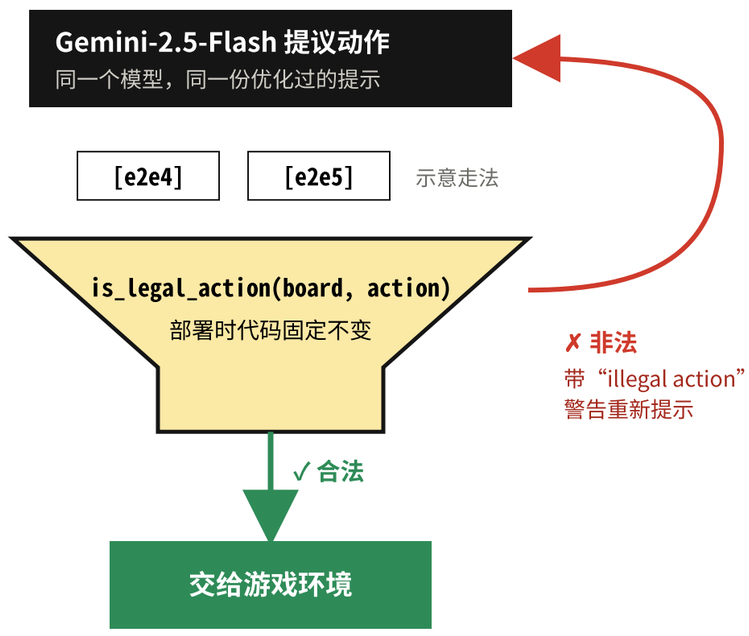
③ **最多取 5 个失败步 → Critic**
把各类错误归并成一段反馈

④ **Refiner 读失败步＋原代码**
写出新一版代码，作为树的新节点

改哪个函数？（§3）

- is_legal_action 说“合法”，环境判非法 → **两个函数都改**
- is_legal_action 说“非法”，确实非法 → **只改 propose_action**

部署阶段：harness-as-action-verifier（本文主体）



闸门代码由 Flash 自己合成，再用来过滤 Flash 自己的提议（作者脚注 2）。实验里去掉了“可用走法”提示，代码只能从棋盘文字和反馈中推断规则。

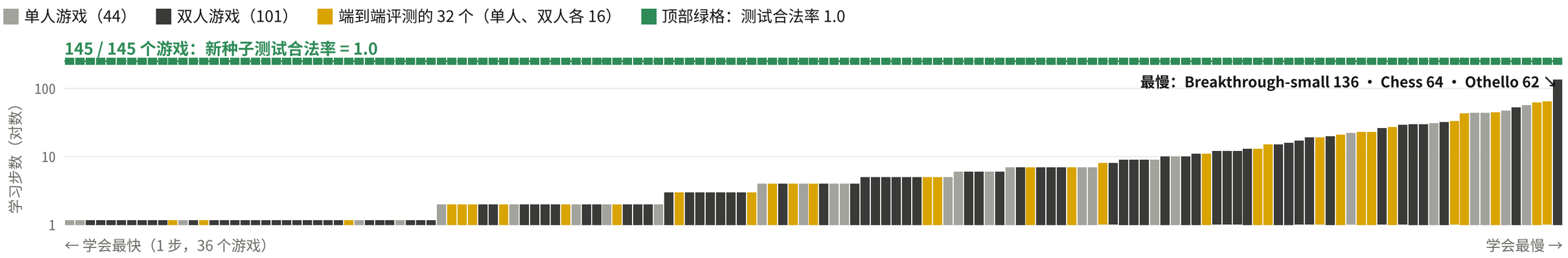
action-filter	action-verifier ← 本文主要结果	harness-as-policy
代码列出全部合法动作，LLM 在其中排序挑选	LLM 先提议，代码验证；不合法就带警告重问。下方 03 的单人 / 双人结果都来自这种形态	整套策略写成代码，推理时不再调用 LLM。另一种产物，结果单列（右下灰卡）

03 / 结果 合法率全部到 1.0；加了闸门的 Flash 在单人和双人游戏上都追平或超过 Pro

§4.1–4.3、附录表 1

145 个游戏、145 条：每条是学到合法率 1.0 所需的步数（附录表 1，按步数排序）

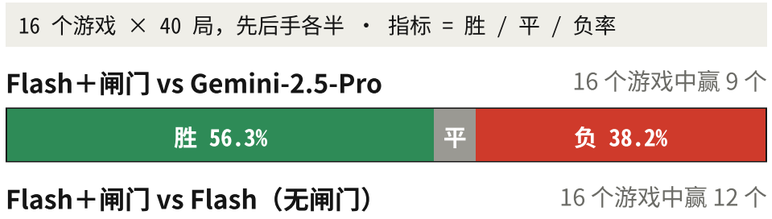
测试：每个游戏 10 个新随机种子 × 1000 步，统计合法动作比例——是**同一游戏的新种子**，不是没见过的游戏。145 个游戏中位数 4 步、均值 10.3（本项目由表 1 计算）；端到端评测的 32 个游戏均值 14.5，19 个不到 10 步。



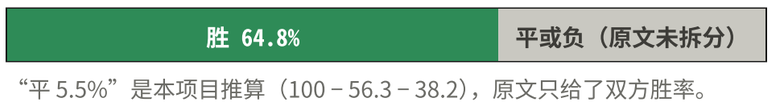
双人游戏：对手是另一个模型

16 个游戏 × 40 局，先后手各半 · 指标 = 胜 / 平 / 负率

Flash＋闸门 vs Gemini-2.5-Pro — 16 个游戏中赢 9 个



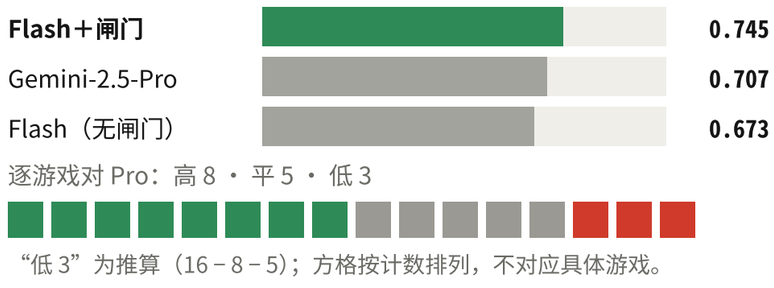
Flash＋闸门 vs Flash（无闸门） — 16 个游戏中赢 12 个



“平 5.5%”是本项目推算（100 − 56.3 − 38.2），原文只给了双方胜率。

单人游戏：比平均奖励，不比胜率

16 个游戏 × 20 局 · 指标 = 平均奖励（0–1）



逐游戏对 Pro：高 8 · 平 5 · 低 3



“低 3”为推算（16 − 8 − 5）；方格按计数排列，不对应具体游戏。

另一种产物：Harness-as-Policy

16 个单人游戏；训练时启发值改为含奖励，最多 256 轮，平均 89.4 轮，启发值 0.939。

纯代码策略 0.870
GPT-5.2-High 0.844（5 次重复）
Gemini-2.5-Pro 0.707
GPT-5.2 0.635（10 次重复）

逐游戏对 GPT-5.2-High：胜 3 · 负 5 · 平 8。推理几乎零成本；两组 GPT-5.2 实验约 $640。

这是整套策略，不是闸门；不能拿来说明闸门的效果

04 / 边界 能说到哪一步

§4–5 · 本项目阅读判断

1. **端到端结果没有方差或置信区间**，每个游戏只有 20 或 40 局；只挑了 145 个游戏中的 32 个。
2. **只和 Gemini 家族比。**闸门部分没有手写 harness、其他合成方法或其他模型族的对照。
3. **无闸门的 Flash 是否也能“非法就重问”，原文没说。**如果不能，部分差距可能来自重试机会本身，而不只是代码规则。
4. **“100% 合法”是同一游戏的新种子**，每个游戏各合成一份代码；可复用的 harness 库是作者列出的未来工作。
5. **78% 败局归因**出自外部赛事报告，本项目没有核对，不作为本文证据。

怎么读这张图

作者结果 合法率、胜率、平均奖励、迭代次数、$640 均为论文报告值。
本项目计算 145 个游戏的中位数 / 均值、条码排序、“平 5.5%”“低 3 个”。
示意 代码树中的版本号与合法率、示意走法；机制图为重绘。

与第一期 Meta-Harness 的区别：那篇优化整套 harness、研究提议者该看什么；本篇只合成一个可验证的部件（合法性检查），按新种子合法率验收。